

windows-image-provisioning

Windows cross-build image provisioning

Sub-package: pkg/crossbuild Backend covered:
WineContainerBackend Forensic anchor: iter-54 of Yole,
2026-05-13

What you are building

The pkg/crossbuild package needs a Linux container image with Wine + JDK 17 + Gradle pre-installed. The image is the runtime that executes the Yole Gradle build inside a Linux host targeting the Windows .msi artifact via Wine's translation layer + the JDK's jpackage tool.

Until this image exists on a given host, pkg/crossbuild's WineContainerBackend.Build() returns a clear actionable error referencing both this document and the #crossbuild-windows-image-provisioning skip-OK ticket so the green-CI / red-Build distinction stays honest.

Provisioning steps (Linux x86_64 host, rootless podman)

```
cd Submodules/Containers/pkg/crossbuild
```

```
# 1. Build the image. ~2-5 minutes on a reasonable network.
```

```
podman build \
  -t ghcr.io/vasic-digital/crossbuild-wine:latest \
  -f windows_wine.Containerfile .
```

```
# 2. Smoke check Wine. Must print "wine-9.x" (or whichever Debian
#    package version this image base ships).
```

```
podman run --rm ghcr.io/vasic-digital/crossbuild-wine:latest wine
  --version
```

```
# 3. Smoke check Gradle. Must print "Gradle 8.11.1".
podman run --rm ghcr.io/vasic-digital/crossbuild-wine:latest
    gradle --version | head -10

# 4. (Optional) Push to the registry so other hosts can pull it
#    without rebuilding. Requires authenticated podman login
#    first.
podman push ghcr.io/vasic-digital/crossbuild-wine:latest
```

Verifying integration with pkg/crossbuild

```
cd Submodules/Containers
go test ./pkg/crossbuild/... -race -count=1
```

Expected: PASS. The integration tests use injected fakes for the container runner so they pass even when the real image is not provisioned. The real-stack Challenge (next section) is the one that exercises the actual image.

Real-stack Challenge

challenges/crossbuild_windows_msi_challenge.sh (Challenge script):

```
bash challenges/crossbuild_windows_msi_challenge.sh \
    --source-dir /path/to/Yole \
    --output-dir /path/to/Yole/releases
```

Expected on Linux x86_64 hosts WITH the image provisioned:

```
OK: produced Yole-Windows-x64-1.0.1-Release-0.0.0.1.1.msi (size N
bytes)
PASS: crossbuild_windows_msi_challenge
```

Expected when the image is NOT yet provisioned:

```
SKIP-OK: #crossbuild-windows-image-provisioning – ghcr.io/vasic-
digital/crossbuild-wine:latest not on host
```

Expected on darwin/aarch64 hosts (Wine-in-Docker not supported):

```
SKIP-OK: #env-darwin-no-wine-container – wine-container backend
requires Linux host
```

A PASS without an actual non-zero .msi is a bluff under
CONST-035 § 11.4 + Lava Sixth Law 6.B. The Challenge enforces
this via `[[ -s "$OUTPUT_MSI" ]]` post-condition.

When this approach hits its limits

Wine-in-Docker cross-builds the .msi reliably for plain Compose Desktop applications. If a Yole release requires Windows-specific behaviour Wine cannot translate (kernel-mode drivers, codec licensing checks, signed-driver loaders), fall back to the QEMUWindowsBackend (sibling Backend; uses pkg/vm/qemu.go to boot a real Windows guest). That path is slower but supports the full Windows API surface. Provisioning the QEMU disk image is documented in docs/crossbuild/qemu-windows-image-provisioning.md (separate file, owed for iter-55+ when the operator commissions it).